



Projeto – Implementação de Sistema de Arquivos EXT4

Descrição:

Implemente estruturas de dados e operações para manipular a imagem (.iso) de um sistema de arquivos EXT4. As operações deverão ser invocadas a partir de um prompt (shell). O shell deve executar as operações a partir da referência do diretório corrente. Considere que o programa shell desenvolvido sempre inicia no raiz (/) da imagem manipulada.

Exemplo:

```
# ext4shell myext4image.img  
ext4shell:[myext4image/] $
```

As operações a serem implementadas são:

- (1) **info**: exibe informações da imagem e do sistema de arquivos.
- (2) **cat <file>**: exibe o conteúdo de um arquivo no formato texto.
- (3) **attr <file | dir>**: exibe os atributos de um arquivo (*file*) ou diretório (*dir*).
- (4) **cd <path>**: altera o diretório corrente para o definido como *path*.
- (5) **ls**: lista os arquivos e diretórios do diretório corrente.
- (6) **testi <inode_number>**: verifica se um inode está livre ou ocupado.
- (7) **testb <block_number>**: verifica se um bloco está livre ou ocupado.
- (8) **export <source_path> <target_path>**: copia arquivo da imagem (*source_path*) para destino (*target_path*) no sistema operacional da máquina.
- (9) **pwd**: exibe o diretório corrente (caminho absoluto).
- (10) **touch <file>**: cria o arquivo *file* com conteúdo vazio.
- (11) **mkdir <dir>**: cria o diretório *dir* vazio.
- (12) **rm <file>**: remove o arquivo *file* do sistema.
- (13) **rmdir <dir>**: remove o diretório *dir*, se estiver vazio.
- (14) **rename <file> <newfilename>**: renomeia arquivo *file* para *newfilename*.

As operações de (1) a (9) envolvem somente a leitura da imagem.

As operações de (10) a (14) envolvem a escrita na imagem.

Linguagens: C/C++

Restrições: Não usar chamadas para funções do sistema (p. ex. `system()`, `exec()`) e estruturas ext4 prontas obtidas de bibliotecas ou da Web.

Simplificações permitidas: as sintaxes dos comandos podem ser simplificadas (p. ex. não é necessário tratar múltiplos diretórios: `rm dir1/dir2/file.txt`). Considere que não teremos diretórios com muitas entradas (arquivos). O sistema sempre usa *extents* e não usa diretórios com *hash tree*. O software deve tratar blocos de tamanho 1KiB, 2KiB e 4KiB.

Equipe: 3 pessoas.

Estrutura do texto: capa, introdução, descrição da atividade, métodos, resultados e discussão, bugs conhecidos, divisão das atividades, conclusões e referências. (*obs.: todos os membros devem trabalhar no código*)



Projeto – Implementação de Sistema de Arquivos EXT4

Submissão: apenas 1 membro do grupo deve submeter no Moodle um único arquivo **tar.gz** contendo relatório (pdf), arquivos fontes, Makefile e readme. Não enviar arquivo binário do código.

Avaliação: qualidade técnica do texto, explicação didática de como foi implementado, código implementado, complexidade da implementação.

1. Documentação: 30%
 - i. relatório (0,20).
 - ii. organização do código com readme, Makefile, nomes de variáveis e comentários (0,10).
2. Codificação: 70%
 - i. operações (1) a (8): 0,4 por operação.
 - ii. operação (9): 0,3.
 - iii. as operações (10) a (14): 0,5 por operação.
 - iv. tratamentos de entrada e saída: 1,0 ponto.
3. Deduções:
 - i. 70% se não compilar;
 - ii. 10% por falhas de execução;
 - iii. 20% se corromper a imagem ISO.

Referências:

- [1] KERNEL WIKI. **Ext4 Disk Layout**. Disponível em https://ext4.wiki.kernel.org/index.php/Ext4_Disk_Layout. Acessado em 09/08/2023.
- [2] THE LINUX KERNEL. **ext4 Data Structures and Algorithms**. Disponível em <https://www.kernel.org/doc/html/latest/filesystems/ext4/index.html>. Acessado em 09/08/2023.
- [3] Dara, Srivathsa. **Understanding Ext4 Disk Layout, Part 1**. Disponível em <https://blogs.oracle.com/linux/post/understanding-ext4-disk-layout-part-1>. Acessado em 07/10/2023.
- [4] Dara, Srivathsa. **Understanding Ext4 Disk Layout, Part 2**. Disponível em <https://blogs.oracle.com/linux/post/understanding-ext4-disk-layout-part-2>. Acessado em 07/10/2023.



Projeto – Implementação de Sistema de Arquivos EXT4

Apêndice A: Comandos e estrutura do volume *myext2image.img*

Gerando imagens ext4 (512MiB com blocos de 4K):

```
# dd if=/dev/zero of=./myext4image.img bs=1024 count=512K  
# mkfs.ext4 -b 4096 ./myext4image.img
```

Verificando a integridade de um sistema ext4:

```
# e2fsck myext4image.img
```

Montando a imagem do volume com ext4:

```
# sudo mount myext4image.img /mnt
```

Depurando a imagem do volume com ext4:

```
# sudo debugfs myext4image.img
```

Estrutura original de arquivos do volume (comando tree via bash):

```
/
├── [1.0K] documentos
│   ├── [1.0K] emptydir
│   ├── [9.2K] alfabeto.txt
│   └── [ 0] vazio.txt
├── [1.0K] imagens
│   ├── [8.1M] one_piece.jpg
│   ├── [391K] saber.jpg
│   └── [11M] toscana_puzzle.jpg
├── [1.0K] livros
│   ├── [1.0K] classicos
│   │   ├── [506K] A Journey to the Centre of the Earth - Jules Verne.txt
│   │   ├── [409K] Dom Casmurro - Machado de Assis.txt
│   │   ├── [861K] Dracula-Bram_Stoker.txt
│   │   ├── [455K] Frankenstein-Mary_Shelley.txt
│   │   └── [232K] The Worderful Wizard of Oz - L. Frank Baum.txt
│   ├── [1.0K] religiosos
│   │   └── [3.9M] Biblia.txt
│   └── [12K] lost+found
└── [ 29] hello.txt
```

Informações de espaço (comando df via bash):

```
Blocos de 4k: X  
Usado: X KiB  
Disponível: X KiB
```

Informações detalhadas do sistema de arquivos :

```
# fsstat myext4image.img  
ou  
# debugfs myext4image.img  
> stats
```

Desmontando a imagem do volume com ext4:

```
# sudo umount /mnt
```